10x Redesign — Production Build Specification

Target Platform	WhatsApp Business API + Apple Messages for Business
LLM Provider	OpenAI ChatGPT (GPT-4o + GPT-4o-mini)
Architecture	3-Plane Model (Gateway / Brain / Hands)
Cost Target	< $0.03 per interaction blended
Primary Language	Python 3.12+ (FastAPI + Temporal)
Database	PostgreSQL 16 + pgvector + Redis 7

February 2026 • CONFIDENTIAL

Table of Contents

1. System Architecture — 3-Plane Model
2. Tech Stack & Infrastructure
3. Database Schema — Complete SQL
4. Pydantic Contracts — Core Data Models
5. API Specification — Endpoints & Contracts
6. Service Architecture — Per-Plane Breakdown
7. Intelligence Engine — 4-Tier Reasoning
8. LLM Integration — Prompts, Routing, Caching
9. Memory Engine — Storage, Retrieval, Learning
10. Autonomy Engine (ACE) — Trust & Approval
11. Channel Integration — WhatsApp & iMessage
12. Tool/Connector Architecture — MCP-Native
13. Proactive Intelligence — Triggers & Scheduling
14. Conversation Design System — Persona & Patterns
15. Context Compiler — Token Budget Management
16. Workflow Orchestration — Temporal Patterns
17. Security Implementation
18. Observability & Monitoring
19. Testing Strategy — Unit to Red-Team
20. Deployment Architecture — AWS Production
21. Onboarding Flow — Zero to Wow
22. Cost Model & Unit Economics
23. Build Plan — Sprint-Level Detail
24. Team Structure & Ownership Matrix

Appendix A. Environment Variables & Config
Appendix B. Error Codes & Taxonomy
Appendix C. WhatsApp Template Library

1. System Architecture — 3-Plane Model

The system is decomposed into three independently deployable **planes**. Each plane has a clear responsibility boundary, its own scaling policy, and can fail without taking down the others.

> **Architecture Principle**
>
> Intelligence can fail safely while the system stays deterministic. The Gateway never crashes because the Brain hallucinated. The Brain never stalls because a Google API timed out in the Hands. Plane isolation is enforced by network policies, separate process groups, and independent health checks.

1.1 Plane Definitions

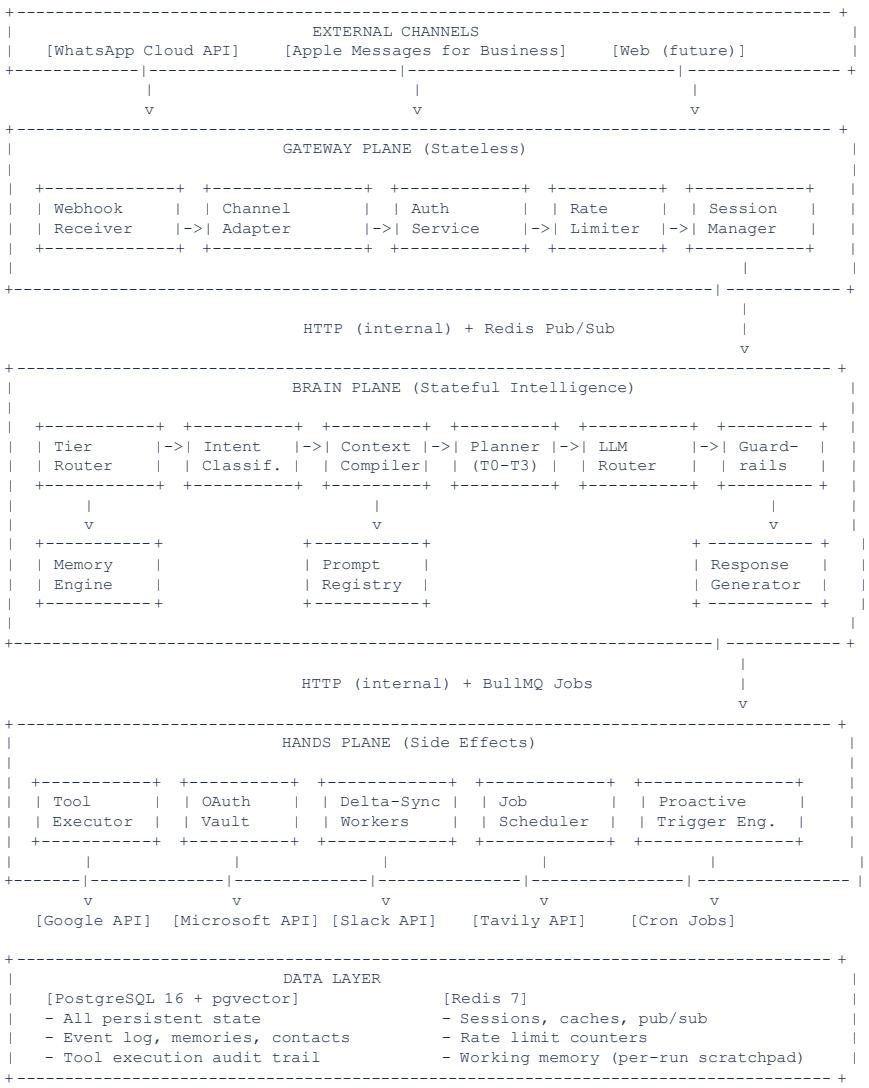
Plane	Responsibility	Key Services	Failure Mode
Gateway	Ingress, auth, session management, rate limiting, channel adaptation, message routing, webhook handling	WhatsApp Webhook Receiver, iMessage Relay, Auth Service, Session Store (Redis), Rate Limiter, Channel Adapter	Returns 503 with queued retry. User sees 'processing' indicator.
Brain	All intelligence: intent classification, tier routing, planning, LLM orchestration, memory retrieval, context compilation, response generation, guardrails	Tier Router, Intent Classifier, Planner (T1-T3), Context Compiler, LLM Router, Memory Engine, Guardrail Validators, Prompt Registry	Degrades to cached responses + read-only mode. User told 'running in fast mode.'
Hands	All external actions: tool execution, OAuth token management, delta-sync, side-effect commits, webhook dispatch, background jobs, proactive triggers	Tool Executor, OAuth Vault, Webhook Handler, Delta-Sync Workers, Job Scheduler (BullMQ), Side-Effect Ledger	Read operations continue via cache. Write operations queued. User told 'actions paused.'

1.2 Inter-Plane Communication

From	To	Protocol	Pattern	Timeout
Gateway	Brain	HTTP/REST (internal)	Sync request/response with SSE for streaming	30s (with 2s typing indicator)
Brain	Hands	HTTP/REST (internal) + BullMQ jobs	Sync for simple tool calls; async job queue for multi-step	10s per tool call; 5min for workflows
Hands	Brain	Redis Pub/Sub + webhook callback	Event notification on tool completion / delta-sync results	N/A (event-driven)
Hands	Gateway	Redis Pub/Sub	Push outbound messages to channel adapter for delivery	N/A (event-driven)
Any	Postgres	Connection pool (asyncpg)	All persistent reads/writes through connection pool	5s query timeout

From	To	Protocol	Pattern	Timeout
Any	Redis	Direct connection (redis-py)	Session state, caches, pub/sub, rate limit counters	1s

1.3 Architecture Diagram



2. Tech Stack & Infrastructure

Every technology choice is evaluated on three axes: **build speed** (can a single engineer set it up in a day?), **operational cost** (monthly $$ at 1K users), and **team expertise** (standard Python/JS skills sufficient?).

Layer	Technology	Version	Purpose	Monthly Cost (1K users)
Language	Python 3.12+	3.12	All backend services. Type hints enforced (mypy strict).	$0
Web Framework	FastAPI	0.115+	Gateway + Brain API. Async, Pydantic-native, OpenAPI auto-gen.	$0
Task Queue	BullMQ (Redis-backed)	5.x	Job scheduling, background tasks, delta-sync triggers. Simple flows.	$0 (uses Redis)
Workflow Engine	Temporal (self-hosted)	1.24+	Complex multi-step workflows only (T3 planning). NOT for simple tasks.	~$50 (container)
Primary DB	PostgreSQL 16	16.x	Source of truth. Event log, users, memories, contacts, tool executions.	~$50 (RDS)
Vector Search	pgvector (PG extension)	0.7+	Semantic memory search. HNSW index. No separate vector DB needed.	$0 (in PG)
Cache / Pub-Sub	Redis 7	7.x	Sessions, semantic cache, rate limits, pub/sub, working memory.	~$25
LLM (Primary)	OpenAI GPT-4o-mini	gpt-4o-mini	80% of calls: intent classification, simple Q&A, summarization, drafts.	~$800
LLM (Complex)	OpenAI GPT-4o	gpt-4o	20% of calls: planning, complex reasoning, multi-step tasks.	~$200
Embeddings	OpenAI text-embedding-3-small	3-small	Memory embeddings, semantic cache keys. 1536 dimensions.	~$20
Messaging	WhatsApp Cloud API	v21.0	Primary channel. Direct Meta API (no Twilio). Free tier: 1K convos/mo.	$0-100
Messaging	Apple Messages for Business	Latest	Secondary channel. Native iMessage with rich interactions.	$0
Auth	Clerk	Latest	Phone-based auth. JWT tokens. Minimal config.	~$25
Object Storage	AWS S3	N/A	Attachments, exports, backups.	~$5
Secrets	AWS Secrets Manager	N/A	OAuth tokens, API keys, encryption keys.	~$10
Observability	Axiom + OpenTelemetry	Latest	Logs, traces, metrics in one platform. OTEL SDK for instrumentation.	~$25
CI/CD	GitHub Actions	N/A	Build, test, deploy pipelines. Contract compatibility checks.	$0 (free tier)
IaC	SST (Serverless Stack)	3.x	AWS infrastructure as TypeScript code. Simpler than Terraform.	$0
Containers	AWS ECS Fargate	N/A	Serverless containers. No GPU nodes. Auto-scaling.	~$150

Layer	Technology	Version	Purpose	Monthly Cost (1K users)
Search (future)	Tavily API or Perplexity	Latest	Web search for research queries. Pay-per-query.	~$30

Total Monthly Infrastructure at 1K Users

Estimated: **$1,390/month** (vs. $8K-$15K in v3.2). At $19.99/user/month subscription = $19,990 revenue. **Gross margin: 93%.**

3. Database Schema — Complete SQL

All SQL is production-ready. Copy-paste into a migration file. Row-Level Security (RLS) is enforced on every table. All timestamps are **timestamptz** (timezone-aware). UUIDs use **gen_random_uuid()** (PG 13+ native).

3.1 Extensions & Enums

```
-- Required extensions CREATE EXTENSION IF NOT EXISTS "pgcrypto"; -- gen_random_uuid() CREATE EXTENSION IF NOT EXISTS "vector"; -- pgvector for embeddings CREATE EXTENSION IF NOT EXISTS "pg_trgm"; -- fuzzy text matching -- Enums CREATE TYPE channel_type AS ENUM ('whatsapp', 'imessage', 'web'); CREATE TYPE message_direction AS ENUM ('inbound', 'outbound'); CREATE TYPE run_state AS ENUM ( 'pending', 'classifying', 'planning', 'executing', 'awaiting_approval', 'completed', 'failed', 'cancelled' ); CREATE TYPE memory_type AS ENUM ('preference', 'fact', 'contact', 'procedure', 'episode'); CREATE TYPE sensitivity_level AS ENUM ('public', 'private', 'sensitive'); CREATE TYPE approval_status AS ENUM ('pending', 'approved', 'denied', 'expired'); CREATE TYPE tool_exec_status AS ENUM ('success', 'failed', 'timeout', 'compensated'); CREATE TYPE risk_level AS ENUM ('none', 'low', 'medium', 'high', 'critical'); CREATE TYPE account_status AS ENUM ('active', 'expired', 'revoked'); CREATE TYPE trigger_type AS ENUM ('schedule', 'event', 'pattern');
```

3.2 Core Tables

users

```
CREATE TABLE users ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), phone_number TEXT UNIQUE NOT NULL, -- E.164 format: +1234567890 display_name TEXT, email TEXT, timezone TEXT DEFAULT 'America/New_York', -- IANA timezone locale TEXT DEFAULT 'en-US', work_hours JSONB DEFAULT '{"start":"09:00","end":"17:00","days":[1,2,3,4,5]}', quiet_hours JSONB DEFAULT '{"start":"22:00","end":"07:00"}', channels JSONB DEFAULT '{}', -- {"whatsapp":{"wa_id":"..."},"imessage":{...}} onboarding_stage INT DEFAULT 0, -- 0=new,1=connected,2=profiled,3=active,4=power autonomy_profile JSONB DEFAULT '{}', -- per-action-class Beta params preferences JSONB DEFAULT '{}', -- quick-access prefs (comm_style, verbosity, etc.) created_at TIMESTAMPTZ DEFAULT now(), updated_at TIMESTAMPTZ DEFAULT now() ); CREATE INDEX idx_users_phone ON users(phone_number); ALTER TABLE users ENABLE ROW LEVEL SECURITY; CREATE POLICY users_isolation ON users USING (id = current_setting('app.user_id')::uuid);
```

conversations

```
CREATE TABLE conversations ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, channel channel_type NOT NULL, state JSONB DEFAULT '{}', -- entity_table, active_tasks, turn_count summary TEXT, -- rolling summary of older turns started_at TIMESTAMPTZ DEFAULT now(), last_active_at TIMESTAMPTZ DEFAULT now(), is_active BOOLEAN DEFAULT true ); CREATE INDEX idx_conversations_user ON conversations(user_id, last_active_at DESC); CREATE INDEX idx_conversations_active ON conversations(user_id, is_active) WHERE is_active = true; ALTER TABLE conversations ENABLE ROW LEVEL SECURITY; CREATE POLICY convos_isolation ON conversations USING (user_id = current_setting('app.user_id')::uuid);
```

messages

```
CREATE TABLE messages ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE, user_id UUID NOT NULL REFERENCES users(id), direction message_direction NOT NULL, content JSONB NOT NULL, -- {text, media_url, buttons[], template_name} intent TEXT, -- classified intent tier INT, -- 0-3, which reasoning tier handled this run_id UUID, -- FK to runs (nullable, set after classification) cost_cents INT DEFAULT 0, -- LLM cost attribution latency_ms INT, -- end-to-end processing time channel_msg_id TEXT, -- provider's message ID for dedup created_at TIMESTAMPTZ DEFAULT now() ); CREATE INDEX idx_messages_convo ON messages(conversation_id, created_at); CREATE INDEX idx_messages_user ON messages(user_id, created_at DESC); CREATE INDEX idx_messages_run ON messages(run_id) WHERE run_id IS NOT NULL; CREATE UNIQUE INDEX idx_messages_channel_dedup ON messages(channel_msg_id) WHERE channel_msg_id IS NOT NULL; ALTER TABLE messages ENABLE ROW LEVEL SECURITY; CREATE POLICY
```

```
messages_isolation ON messages USING (user_id = current_setting('app.user_id')::uuid);
```

runs

```
CREATE TABLE runs ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES
users(id), conversation_id UUID NOT NULL REFERENCES conversations(id), parent_run_id UUID REFERENCES
runs(id), -- for sub-tasks intent TEXT NOT NULL, tier INT NOT NULL DEFAULT 1, -- 0-3 plan JSONB DEFAULT
'[]', -- [{step_id, tool, args, status, result, cost}] state run_state NOT NULL DEFAULT 'pending',
envelope JSONB NOT NULL, -- {tools_allowed[], budget_cents, sensitivity_cap, expires_at} error JSONB, --
{code, message, recoverable, user_message} total_cost_cents INT DEFAULT 0, total_latency_ms INT DEFAULT 0,
created_at TIMESTAMPTZ DEFAULT now(), completed_at TIMESTAMPTZ ); CREATE INDEX idx_runs_user_state ON
runs(user_id, state); CREATE INDEX idx_runs_convo ON runs(conversation_id); CREATE INDEX idx_runs_parent
ON runs(parent_run_id) WHERE parent_run_id IS NOT NULL; ALTER TABLE runs ENABLE ROW LEVEL SECURITY; CREATE
POLICY runs_isolation ON runs USING (user_id = current_setting('app.user_id')::uuid);
```

memories

```
CREATE TABLE memories ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES
users(id) ON DELETE CASCADE, type memory_type NOT NULL, category TEXT NOT NULL, -- scheduling,
communication, finance, work, etc. key TEXT NOT NULL, -- semantic key for dedup: 'preference:meeting_time'
content JSONB NOT NULL, -- {value, context, reasoning} embedding vector(1536), -- for semantic search
confidence FLOAT DEFAULT 0.8 CHECK (confidence BETWEEN 0.0 AND 1.0), sensitivity sensitivity_level DEFAULT
'private', valid_from TIMESTAMPTZ DEFAULT now(), valid_until TIMESTAMPTZ, -- NULL = no expiry
source_run_id UUID REFERENCES runs(id), -- provenance version INT DEFAULT 1, is_active BOOLEAN DEFAULT
true, created_at TIMESTAMPTZ DEFAULT now(), updated_at TIMESTAMPTZ DEFAULT now(), UNIQUE(user_id, key,
version) ); -- HNSW index for fast semantic search (cosine distance) CREATE INDEX idx_memories_embedding
ON memories USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64); CREATE INDEX
idx_memories_user_type ON memories(user_id, type, category) WHERE is_active = true; CREATE INDEX
idx_memories_user_key ON memories(user_id, key) WHERE is_active = true; ALTER TABLE memories ENABLE ROW
LEVEL SECURITY; CREATE POLICY memories_isolation ON memories USING (user_id =
current_setting('app.user_id')::uuid);
```

contacts

```
CREATE TABLE contacts ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES
users(id) ON DELETE CASCADE, name TEXT NOT NULL, email TEXT, phone TEXT, relationship TEXT, -- colleague,
friend, family, service_provider organization TEXT, title TEXT, trust_score FLOAT DEFAULT 0.5 CHECK
(trust_score BETWEEN 0.0 AND 1.0), interaction_count INT DEFAULT 0, metadata JSONB DEFAULT '{}', --
{timezone, notes, tags[], source} last_interacted_at TIMESTAMPTZ, created_at TIMESTAMPTZ DEFAULT now(),
updated_at TIMESTAMPTZ DEFAULT now() ); CREATE INDEX idx_contacts_user ON contacts(user_id); CREATE INDEX
idx_contacts_name_trgm ON contacts USING gin (name gin_trgm_ops); -- fuzzy search CREATE INDEX
idx_contacts_email ON contacts(user_id, email) WHERE email IS NOT NULL; CREATE INDEX idx_contacts_trust ON
contacts(user_id, trust_score DESC); ALTER TABLE contacts ENABLE ROW LEVEL SECURITY; CREATE POLICY
contacts_isolation ON contacts USING (user_id = current_setting('app.user_id')::uuid);
```

tool_executions

```
CREATE TABLE tool_executions ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), run_id UUID NOT NULL
REFERENCES runs(id), user_id UUID NOT NULL REFERENCES users(id), tool_name TEXT NOT NULL, -- e.g.,
google_calendar.create_event input_hash TEXT NOT NULL, -- SHA-256 of canonical input input JSONB NOT NULL,
-- tool input (PII fields redacted in logs) output JSONB, -- tool output status tool_exec_status NOT NULL
DEFAULT 'success', error JSONB, -- {code, message, retryable} idempotency_key TEXT UNIQUE NOT NULL, --
prevents duplicate execution risk_level risk_level NOT NULL DEFAULT 'none', approved_by UUID REFERENCES
approvals(id), -- if approval was required cost_cents INT DEFAULT 0, latency_ms INT NOT NULL, created_at
TIMESTAMPTZ DEFAULT now() ); CREATE INDEX idx_tool_exec_run ON tool_executions(run_id); CREATE INDEX
idx_tool_exec_user ON tool_executions(user_id, created_at DESC); CREATE INDEX idx_tool_exec_idemp ON
tool_executions(idempotency_key); ALTER TABLE tool_executions ENABLE ROW LEVEL SECURITY; CREATE POLICY
tool_exec_isolation ON tool_executions USING (user_id = current_setting('app.user_id')::uuid);
```

approvals

```
CREATE TABLE approvals ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), run_id UUID NOT NULL REFERENCES runs(id), user_id UUID NOT NULL REFERENCES users(id), action_summary TEXT NOT NULL, -- "Send email to john@acme.com" risk_explanation TEXT NOT NULL, -- "New contact not seen before" proposed_action JSONB NOT NULL, -- full tool call details alternatives JSONB DEFAULT '[]', -- [{label, action}] status approval_status DEFAULT 'pending', channel channel_type NOT NULL, channel_msg_id TEXT, -- message ID of approval request responded_at TIMESTAMPTZ, expires_at TIMESTAMPTZ NOT NULL, created_at TIMESTAMPTZ DEFAULT now() ); CREATE INDEX idx_approvals_user_pending ON approvals(user_id, status) WHERE status = 'pending'; CREATE INDEX idx_approvals_run ON approvals(run_id); ALTER TABLE approvals ENABLE ROW LEVEL SECURITY; CREATE POLICY approvals_isolation ON approvals USING (user_id = current_setting('app.user_id')::uuid);
```

connected_accounts

```
CREATE TABLE connected_accounts ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, provider TEXT NOT NULL, -- google, microsoft, slack provider_account_id TEXT, -- provider's user ID access_token_encrypted BYTEA NOT NULL, -- AES-256-GCM encrypted refresh_token_encrypted BYTEA, token_expires_at TIMESTAMPTZ, scopes TEXT[] NOT NULL, sync_cursors JSONB DEFAULT '{}', -- {calendar: "token...", email: "token..."} last_synced_at TIMESTAMPTZ, status account_status DEFAULT 'active', created_at TIMESTAMPTZ DEFAULT now(), updated_at TIMESTAMPTZ DEFAULT now(), UNIQUE(user_id, provider) ); ALTER TABLE connected_accounts ENABLE ROW LEVEL SECURITY; CREATE POLICY accounts_isolation ON connected_accounts USING (user_id = current_setting('app.user_id')::uuid);
```

proactive_triggers

```
CREATE TABLE proactive_triggers ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE, type trigger_type NOT NULL, name TEXT NOT NULL, -- "morning_briefing", "pre_meeting_prep" trigger_condition JSONB NOT NULL, -- {cron?, event_type?, pattern?} action_template JSONB NOT NULL, -- {tool, args_template, message_template} confidence_threshold FLOAT DEFAULT 0.7, enabled BOOLEAN DEFAULT true, max_daily_fires INT DEFAULT 3, last_fired_at TIMESTAMPTZ, fire_count INT DEFAULT 0, success_count INT DEFAULT 0, user_dismissed_count INT DEFAULT 0, -- tracks annoyance created_at TIMESTAMPTZ DEFAULT now() ); CREATE INDEX idx_triggers_user ON proactive_triggers(user_id, enabled) WHERE enabled = true; ALTER TABLE proactive_triggers ENABLE ROW LEVEL SECURITY; CREATE POLICY triggers_isolation ON proactive_triggers USING (user_id = current_setting('app.user_id')::uuid);
```

semantic_cache

```
CREATE TABLE semantic_cache ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), user_id UUID NOT NULL REFERENCES users(id), query_embedding vector(1536) NOT NULL, query_text TEXT NOT NULL, response JSONB NOT NULL, -- cached LLM response model TEXT NOT NULL, tier INT NOT NULL, hit_count INT DEFAULT 0, created_at TIMESTAMPTZ DEFAULT now(), expires_at TIMESTAMPTZ NOT NULL -- TTL-based expiry ); CREATE INDEX idx_sem_cache_embedding ON semantic_cache USING hnsw (query_embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64); CREATE INDEX idx_sem_cache_expiry ON semantic_cache(expires_at);
```

4. Pydantic Contracts — Core Data Models

All data flowing between services is validated by Pydantic v2 models in **strict mode**. These contracts are the single source of truth. OpenAPI schemas, tool definitions, and database serialization all derive from these models.

4.1 Core Envelopes & Messages

```
from pydantic import BaseModel, Field, field_validator from enum import Enum from uuid import UUID, uuid4
from datetime import datetime from typing import Optional class Channel(str, Enum): WHATSAPP = "whatsapp"
IMESSAGE = "imessage" WEB = "web" class RunState(str, Enum): PENDING = "pending" CLASSIFYING =
"classifying" PLANNING = "planning" EXECUTING = "executing" AWAITING_APPROVAL = "awaiting_approval"
COMPLETED = "completed" FAILED = "failed" CANCELLED = "cancelled" class RiskLevel(str, Enum): NONE =
"none" LOW = "low" MEDIUM = "medium" HIGH = "high" CRITICAL = "critical" class Tier(int, Enum): INSTANT =
0 # Pattern match, no LLM SINGLE_SHOT = 1 # One LLM call + tool REACT = 2 # ReAct loop, max 5 iterations
MULTI_STEP = 3 # Full planning + critic # --- Capability Envelope --- class CapabilityEnvelope(BaseModel):
"""Bound to every run. Defines what the agent CAN do."""  envelope_id: UUID = Field(default_factory=uuid4)
user_id: UUID tools_allowed: list[str] # ["google_calendar.*", "google_gmail.read"] budget_cents: int =
100 # max cost for this run sensitivity_cap: str = "private" # max sensitivity level accessible risk_cap:
RiskLevel = RiskLevel.MEDIUM expires_at: datetime policy_version: str # hash of policy bundle # ---
Inbound Message --- class InboundMessage(BaseModel): channel: Channel channel_msg_id: str # dedup key
user_phone: str # E.164 text: Optional[str] = None media_url: Optional[str] = None media_type:
Optional[str] = None timestamp: datetime context: Optional[dict] = None # channel-specific metadata # ---
Outbound Message --- class OutboundMessage(BaseModel): channel: Channel user_id: UUID text: Optional[str]
= None buttons: Optional[list[dict]] = None # [{id, title}] max 3 for WhatsApp template: Optional[dict] =
None # {name, language, components[]} media_url: Optional[str] = None typing_indicator: bool = False
```

4.2 Intent Classification & Tier Routing

```
class IntentClassification(BaseModel): """Output of the Tier Router. Determines how the message is
processed."""  intent: str # "schedule_meeting", "check_calendar", "draft_email", etc. tier: Tier
confidence: float = Field(ge=0.0, le=1.0) required_tools: list[str] = [] # tools the planner will need
is_continuation: bool = False # modifying an in-progress task? continuation_run_id: Optional[UUID] = None
entities: dict = {} # extracted entities: {person: "John", time: "tomorrow 3pm"} class
TierRoutingConfig(BaseModel): """Per-intent routing rules."""  intent_pattern: str # regex or exact match
default_tier: Tier escalation_rules: dict = {} # {"on_tool_failure": Tier.REACT, "on_ambiguity":
Tier.REACT} required_tools: list[str] = [] max_cost_cents: int = 50
```

4.3 Tool Contracts

```
class ToolSpec(BaseModel): """Every connector tool implements this contract."""  name: str #
"google_calendar.create_event" description: str # for LLM function calling input_schema: dict # JSON
Schema (auto-generated from Pydantic model) output_schema: dict risk_level: RiskLevel = RiskLevel.NONE
is_reversible: bool = False requires_approval_above: RiskLevel = RiskLevel.MEDIUM timeout_ms: int = 10000
retry_policy: dict = {"max_retries": 2, "backoff": "exponential"} rate_limit_per_min: int = 30 tags:
list[str] = [] # ["calendar", "write", "google"] class ToolCall(BaseModel): """A request to execute a
tool."""  tool_name: str arguments: dict idempotency_key: str # stable hash of (run_id + step_id + tool +
args) envelope_id: UUID run_id: UUID class ToolResult(BaseModel): """Result of a tool execution."""
tool_name: str status: str # "success" | "failed" | "timeout" output: Optional[dict] = None error:
Optional[dict] = None # {code, message, retryable} latency_ms: int cost_cents: int = 0
```

4.4 Memory Contracts

```
class MemoryEntry(BaseModel): """A single memory in the user's knowledge base."""  id: UUID =
Field(default_factory=uuid4) type: str # preference | fact | contact | procedure | episode category: str #
scheduling, communication, finance, work key: str # semantic dedup key content: dict # {value: ...,
context: ..., reasoning: ...} confidence: float = Field(default=0.8, ge=0.0, le=1.0) sensitivity: str =
"private" source_run_id: Optional[UUID] = None class MemoryQuery(BaseModel): """Request to retrieve
```

```
relevant memories."""" user_id: UUID query_text: str intent: Optional[str] = None # pre-filter by relevance
types: Optional[list[str]] = None # filter memory types max_results: int = 10 min_confidence: float = 0.3
class MemoryRetrievalResult(BaseModel): memories: list[MemoryEntry] total_tokens: int # for context budget
tracking retrieval_latency_ms: int
```

5. API Specification — Endpoints & Contracts

The API is intentionally thin. Webhooks receive messages, the Brain processes them, and the Hands take action. The API exists for: (1) channel webhooks, (2) user-facing operations (memory, settings, approvals), and (3) internal health/admin.

5.1 Webhook Endpoints (Gateway Plane)

Endpoint	Method	Purpose	Auth	Response
/webhooks/whatsapp	POST	Receive WhatsApp messages. Validate X-Hub-Signature-256 header. Acknowledge within 5s, process async.	HMAC signature	200 OK (immediate)
/webhooks/whatsapp	GET	WhatsApp webhook verification (hub.challenge handshake)	Verify token	hub.challenge value
/webhooks/imessage	POST	Receive Apple Messages for Business events	Apple cert validation	200 OK (immediate)
/webhooks/oauth/callback	GET	OAuth callback for Google/Microsoft account linking	State parameter + PKCE	302 redirect to success page

5.2 User-Facing API

Endpoint	Method	Purpose	Request Body	Response
/api/v1/messages	POST	Send a message (for web channel or testing)	{text, channel, media_url?}	{message_id, status}
/api/v1/runs/{id}	GET	Get run status, plan, cost	N/A	{run_id, state, plan[], cost_cents, latency_ms}
/api/v1/approvals/{id}	POST	Approve or deny a pending action	{decision: approve|deny}	{approval_id, status, next_action}
/api/v1/memory	GET	List user's memories (paginated, filterable)	?type=preference&q=meeting	{memories[], total, page}
/api/v1/memory/{id}	PUT	Update a memory (user correction)	{content, confidence?}	{memory_id, updated}
/api/v1/memory/{id}	DELETE	Delete a memory	N/A	{deleted: true}
/api/v1/settings	GET/PUT	User preferences (quiet hours, autonomy, etc.)	JSONB partial update	{settings}
/api/v1/accounts	GET	List connected accounts and sync status	N/A	{accounts[]}
/api/v1/accounts/{id}	DELETE	Disconnect an account (revoke tokens)	N/A	{revoked: true}

5.3 Internal/Admin API

Endpoint	Method	Purpose
/internal/health	GET	Shallow health check (process alive)
/internal/health/deep	GET	Deep health: Postgres, Redis, LLM API, each connector
/internal/metrics	GET	Prometheus-format metrics for observability
/internal/runs/{id}/replay	POST	Replay a run from event log (debugging)
/internal/cache/flush	POST	Flush semantic cache for a user (admin)
/internal/triggers/fire	POST	Manually fire a proactive trigger (testing)

6. Service Architecture — Per-Plane Breakdown

Each plane is a set of co-located services sharing a process group. Services within a plane communicate via function calls. Services across planes communicate via HTTP or Redis pub/sub.

6.1 Gateway Plane Services

Service	Responsibility	Implementation	Dependencies
WebhookReceiver	Accepts inbound webhooks from WhatsApp/iMessage. Validates signatures. Deduplicates by channel_msg_id. Publishes to internal message queue.	FastAPI route handlers	Channel credentials (HMAC secrets, certs)
ChannelAdapter	Translates between internal OutboundMessage format and channel-native APIs. Handles WhatsApp button limits (max 3), message splitting (max 4096 chars), template rendering, iMessage rich links.	Strategy pattern per channel	Channel API tokens
AuthService	Resolves phone number to user_id. Creates user on first contact. Manages session JWT tokens for web channel.	Clerk SDK integration	Clerk API key
SessionManager	Maintains conversation state in Redis. Creates new conversation after 30min inactivity. Manages entity resolution table (coreference: 'he' = 'John Smith').	Redis hash per session	Redis connection
RateLimiter	Per-user rate limiting: 60 msgs/min default. Gradual degradation (slower responses), not hard cutoff. Abuse detection: flag accounts sending >200 msgs/hour.	Redis sliding window	Rate limit config

6.2 Brain Plane Services

Service	Responsibility	Implementation	Dependencies
TierRouter	Classifies every inbound message into T0-T3. Stage 1: regex pre-filter (greetings, thanks). Stage 2: GPT-4o-mini with function calling. Stage 3: runtime escalation on failure.	Rules + LLM classifier	OpenAI API, intent config
ContextCompiler	Assembles the LLM context window within tier-specific token budgets. Retrieves memories, injects tool schemas, compresses conversation history, formats user profile.	Token-counting pipeline	Memory Engine, Prompt Registry, tiktoken
Planner	T0: template response. T1: single function call. T2: ReAct loop (max 5 iterations). T3: plan generation + critic review + checkpointed execution.	Tiered strategy pattern	LLM Router, Tool Registry

Service	Responsibility	Implementation	Dependencies
LLMRouter	Routes LLM calls to the right model. GPT-4o-mini for T0/T1. GPT-4o for T2/T3. Handles retries, fallbacks, streaming. Tracks token usage and cost.	OpenAI SDK with retry	OpenAI API key, model config
MemoryEngine	Retrieval: multi-signal ranking (structured lookup + pgvector semantic search + recency decay). Writing: extract facts from run, dedup, store with confidence.	Postgres + pgvector queries	Postgres, embeddings API
GuardrailValidators	Input: sanitize user message (strip injection attempts). Output: validate LLM response against Pydantic schema. Content: OpenAI moderation API check.	Pydantic + moderation API	OpenAI Moderation API
PromptRegistry	Versioned prompts stored in DB + git. A/B testing via feature flags. Per-prompt performance tracking (quality score, cost, latency).	DB table + git sync	Postgres, feature flag service

6.3 Hands Plane Services

Service	Responsibility	Implementation	Dependencies
ToolExecutor	Executes tool calls. Validates input schema. Enforces idempotency (check idempotency_key before execution). Records execution in tool_executions table. Respects rate limits.	Per-tool adapter + common executor	Connector SDKs, Postgres
OAuthVault	Manages OAuth tokens. AES-256-GCM encryption at rest. Auto-refresh before expiry. Revocation on disconnect. Scoped access (calendar-only vs. full).	AWS Secrets Manager + DB	AWS SM, Postgres
DeltaSyncWorkers	Periodically sync data from connected accounts. Calendar: every 2min. Email: every 5min. Contacts: daily. Uses sync cursors for incremental fetch.	BullMQ recurring jobs	Google/Microsoft APIs, Postgres
JobScheduler	Manages all background work: delta-sync, proactive triggers, memory consolidation, cache cleanup. BullMQ with Redis backing. Dead-letter queue for failures.	BullMQ	Redis
ProactiveTriggerEngine	Evaluates proactive trigger conditions. Computes Value > Interruption score. Respects quiet hours and frequency caps. Sends via Gateway.	Cron + event listeners	Postgres, Gateway API

7. Intelligence Engine — 4-Tier Reasoning

The reasoning engine is the core differentiator. It matches **reasoning effort to task complexity**, ensuring simple tasks are fast and cheap while complex tasks get the full planning treatment.

7.1 Tier Definitions & Processing Pipelines

Tier 0: Instant (15-20% of traffic)

Pattern-matched responses with zero LLM calls.

```
1. Regex/keyword match on user message 2. Lookup response template 3. Variable substitution (user name, time) 4. Return immediately
```

Cost: < $0.001 | **Example:** 'Good morning' -> 'Morning, Albert! You have 4 meetings today. First is at 10am with Sarah.'

Tier 1: Single-Shot (50-60% of traffic)

One GPT-4o-mini call with function calling. Direct tool execution.

```
1. Context compile (3,400 tokens max) 2. GPT-4o-mini with tool schemas 3. Execute tool call if returned 4. Format and return result
```

Cost: $0.002-$0.008 | **Example:** 'What's on my calendar tomorrow?' -> [call google_calendar.list_events] -> 'Tomorrow you have 3 meetings: ...'

Tier 2: ReAct Loop (20-25% of traffic)

GPT-4o with iterative Thought-Action-Observation. Max 5 iterations.

```
1. Context compile (11,000 tokens max) 2. GPT-4o generates Thought + Action 3. Execute tool, get Observation 4. Loop until task complete or max iterations 5. If stuck after 5 iterations, escalate to T3 or ask user
```

Cost: $0.02-$0.08 | **Example:** 'Draft a reply to Sarah's email about the budget' -> [search_email] -> [get_memory: Sarah context] -> [draft_email] -> 'Here's a draft: ...'

Tier 3: Multi-Step Plan (3-5% of traffic)

Full plan generation with critic review and checkpointed execution.

```
1. Context compile (23,500 tokens max) 2. GPT-4o generates plan: [{step, tool, args, deps}] 3. Critic call reviews plan for safety + completeness 4. Execute steps with checkpoints 5. If step fails, re-plan from checkpoint 6. Max 3 re-plan attempts before escalating to user
```

Cost: $0.10-$0.30 | **Example:** 'Prepare for my meeting with the investors on Thursday' -> [plan: get_calendar_event, search_emails, get_contacts, draft_briefing, set_reminder] -> execute with progress updates

8. LLM Integration — Prompts, Routing, Caching

8.1 System Prompt Structure

Every LLM call uses a structured system prompt assembled by the Context Compiler. The prompt has 5 sections, each with a token budget.

```
# System Prompt Template (assembled per-call by Context Compiler) SYSTEM_PROMPT = """ ## Identity You are Exec, a personal executive assistant. You are competent, warm, concise, and proactive. You communicate like an exceptional EA who has worked with {user_name} for 3 years. ## Communication Rules - Lead with the answer, then context if needed - Never use filler: no "Certainly!", "Of course!", "I'd be happy to..." - Keep responses under 200 words unless the user asks for detail - For WhatsApp: max 500 chars per message. Split long responses. - Own mistakes directly: "I got the date wrong -- here's the corrected version." ## Current Context - User: {user_name} ({timezone}) - Current time: {current_time} - Channel: {channel} (constraints: {channel_constraints}) - Active conversation: {conversation_summary} ## User Preferences {formatted_preferences} ## Relevant Memories {formatted_memories} ## Available Tools {formatted_tool_schemas} """
```

8.2 Model Routing Rules

Call Type	Model	Temperature	Max Tokens	Structured Output?
Intent classification	gpt-4o-mini	0.0	200	Yes - IntentClassification schema
T1 simple Q&A	gpt-4o-mini	0.3	500	No - free text
T1 tool selection	gpt-4o-mini	0.0	300	Yes - function calling
T2 ReAct reasoning	gpt-4o	0.2	1000	Yes - Thought/Action/Observation
T3 plan generation	gpt-4o	0.3	2000	Yes - Plan schema
T3 plan critic	gpt-4o	0.1	500	Yes - CriticResult schema
Memory extraction	gpt-4o-mini	0.0	300	Yes - MemoryEntry[] schema
Coreference resolution	gpt-4o-mini	0.0	200	Yes - entity table update
Email draft	gpt-4o	0.4	1000	No - free text
Morning briefing	gpt-4o-mini	0.3	500	No - free text

8.3 Semantic Cache Strategy

Before making any LLM call (except intent classification), check the semantic cache. If a similar query was answered recently for this user, reuse the response.

```
async def check_semantic_cache(user_id: UUID, query: str, tier: int) -> Optional[dict]: """Check if a similar query was recently answered.""" query_embedding = await get_embedding(query) # Search cache with cosine similarity > 0.95 result = await db.fetchrow(""" SELECT response, 1 - (query_embedding <=> $1) as similarity FROM semantic_cache WHERE user_id = $2 AND tier = $3 AND expires_at > now() AND 1 - (query_embedding <=> $1) > 0.95 ORDER BY similarity DESC LIMIT 1 """, query_embedding, user_id, tier) if result: # Update hit count await db.execute("UPDATE semantic_cache SET hit_count = hit_count + 1 WHERE id = $1", result['id']) return result['response'] return None # TTL per tier: # T0: N/A (no LLM call) # T1: 15 minutes (calendar data changes frequently) # T2: 5 minutes (context-dependent) # T3: 1 minute (complex, context-dependent)
```

9. Memory Engine — Storage, Retrieval, Learning

The memory engine is what makes the agent get smarter over time. It has three operations: **Retrieve** (get relevant memories for a task), **Write** (store new information after a run), and **Consolidate** (clean up, merge, and decay nightly).

9.1 Retrieval Pipeline (Called by Context Compiler)

```
async def retrieve_memories(query: MemoryQuery) -> MemoryRetrievalResult: """Multi-signal memory retrieval pipeline.""" results = [] # Stage 1: Structured lookup (for preference/fact types) if query.intent: category = intent_to_category(query.intent) # "schedule_meeting" -> "scheduling" structured = await db.fetch(""" SELECT * FROM memories WHERE user_id = $1 AND type IN ('preference', 'fact') AND category = $2 AND is_active = true AND confidence >= $3 ORDER BY confidence DESC LIMIT 5 """, query.user_id, category, query.min_confidence) results.extend(structured) # Stage 2: Contact resolution (if entities contain person names) if 'person' in query.entities: contacts = await db.fetch(""" SELECT * FROM contacts WHERE user_id = $1 AND similarity(name, $2) > 0.3 ORDER BY similarity(name, $2) DESC, interaction_count DESC LIMIT 3 """, query.user_id, query.entities['person']) results.extend(contact_to_memory(c) for c in contacts) # Stage 3: Semantic search (for episodic memories) query_embedding = await get_embedding(query.query_text) semantic = await db.fetch(""" SELECT *, 1 - (embedding <=> $1) as sim, EXTRACT(EPOCH FROM (now() - created_at)) / 86400.0 as age_days FROM memories WHERE user_id = $2 AND is_active = true AND confidence >= $3 AND ($4::text[] IS NULL OR type = ANY($4)) ORDER BY (1 - (embedding <=> $1)) * 0.7 + -- semantic relevance (1.0 / (1.0 + EXTRACT(EPOCH FROM (now() - created_at)) / 86400.0)) * 0.2 + -- recency confidence * 0.1 -- confidence DESC LIMIT $5 """, query_embedding, query.user_id, query.min_confidence, query.types, query.max_results) results.extend(semantic) # Stage 4: Deduplicate and rank seen_keys = set() final = [] for m in results: if m['key'] not in seen_keys: seen_keys.add(m['key']) final.append(m) # Format and count tokens formatted = format_memories_for_context(final[:query.max_results]) return MemoryRetrievalResult( memories=final[:query.max_results], total_tokens=count_tokens(formatted), retrieval_latency_ms=elapsed_ms() )
```

9.2 Memory Write Pipeline (After Every Run)

```
async def extract_and_store_memories(run: Run, messages: list[Message]): """Extract learnings from a completed run and store as memories.""" # Use GPT-4o-mini to extract facts from the conversation extraction_prompt = f""" Analyze this conversation and extract any new information about the user. Return a JSON array of memories to store. Conversation: {format_messages(messages)} Run outcome: {run.state} (intent: {run.intent}) For each memory, provide: - type: "preference" | "fact" | "episode" - category: scheduling | communication | finance | work | personal | etc. - key: semantic dedup key (e.g., "preference:meeting_time") - content: {{value: ..., context: ..., reasoning: ...}} - confidence: 0.0-1.0 (1.0 for explicit statements, 0.5 for inferences) - sensitivity: "public" | "private" | "sensitive" Only extract genuinely new or updated information. Do NOT extract: - Information already known (check existing memories list) - Trivial facts (user said "ok", "thanks") - One-time contextual details unlikely to be useful again """ new_memories = await llm_call(extraction_prompt, model="gpt-4o-mini", schema=MemoryEntry) for memory in new_memories: # Check for conflicts with existing memories existing = await db.fetchrow( "SELECT * FROM memories WHERE user_id = $1 AND key = $2 AND is_active = true", run.user_id, memory.key ) if existing: if memory.confidence > existing['confidence']: # New memory wins - archive old, store new await db.execute( "UPDATE memories SET is_active = false WHERE id = $1", existing['id'] ) await store_memory(memory, run_id=run.id, version=existing['version'] + 1) # else: existing is more confident, skip else: await store_memory(memory, run_id=run.id, version=1) # Always store an episode summary episode = MemoryEntry( type="episode", category=intent_to_category(run.intent), key=f"episode:{run.id}", content={"summary": summarize_run(run), "outcome": run.state, "cost": run.total_cost_cents}, confidence=1.0, source_run_id=run.id ) await store_memory(episode, run_id=run.id)
```

10. Autonomy Engine (ACE) — Trust & Approval

ACE determines whether the agent executes autonomously or asks for permission. It uses per-(user, action_class) Beta distributions that update with every approval/denial.

10.1 Action Classes & Cold-Start Priors

```
# Default autonomy profile (stored in users.autonomy_profile JSONB) DEFAULT_AUTONOMY_PROFILE = { "read_data": {"a": 10, "b": 1, "threshold": 0.0}, # always auto "search_research": {"a": 10, "b": 1, "threshold": 0.0}, # always auto "draft_content": {"a": 2, "b": 3, "threshold": 0.7}, # cautious start "schedule_event": {"a": 2, "b": 3, "threshold": 0.75}, # cautious start "send_known_contact": {"a": 1, "b": 4, "threshold": 0.8}, # very cautious "send_new_contact": {"a": 1, "b": 9, "threshold": 1.0}, # always ask "financial_action": {"a": 1, "b": 9, "threshold": 1.0}, # always ask + step-up "delete_cancel": {"a": 1, "b": 4, "threshold": 0.85}, # very cautious } async def should_auto_execute(user_id: UUID, action_class: str) -> tuple[bool, float]: """Check if the agent should auto-execute or ask for approval.""" profile = await get_autonomy_profile(user_id) params = profile.get(action_class, DEFAULT_AUTONOMY_PROFILE.get(action_class)) if not params: return False, 0.0 # unknown action class = always ask # Expected value of Beta distribution a, b, threshold = params['a'], params['b'], params['threshold'] autonomy_score = a / (a + b) return autonomy_score >= threshold, autonomy_score async def update_autonomy(user_id: UUID, action_class: str, approved: bool): """Update Beta params after user approves or denies an action.""" profile = await get_autonomy_profile(user_id) params = profile.get(action_class, DEFAULT_AUTONOMY_PROFILE.get(action_class)) if approved: params['a'] += 1 else: params['b'] += 1 profile[action_class] = params await db.execute( "UPDATE users SET autonomy_profile = $1 WHERE id = $2", json.dumps(profile), user_id )
```

10.2 Approval Flow (WhatsApp-Native)

```
async def request_approval(run_id: UUID, tool_call: ToolCall, risk_explanation: str): """Send an approval request via the user's active channel.""" user = await get_user(tool_call.user_id) channel = get_active_channel(user) # Create approval record approval = await db.fetchrow(""" INSERT INTO approvals (run_id, user_id, action_summary, risk_explanation, proposed_action, status, channel, expires_at) VALUES ($1, $2, $3, $4, $5, 'pending', $6, now() + interval '10 minutes') RETURNING * """, run_id, user.id, summarize_action(tool_call), risk_explanation, tool_call.dict(), channel) # Send channel-native approval message if channel == Channel.WHATSAPP: await send_whatsapp_interactive(user, { "type": "button", "body": {"text": f"{summarize_action(tool_call)}\n\n{risk_explanation}"}, "action": { "buttons": [ {"type": "reply", "reply": {"id": f"approve_{approval['id']}", "title": "Approve"}}, {"type": "reply", "reply": {"id": f"edit_{approval['id']}", "title": "Edit"}}, {"type": "reply", "reply": {"id": f"deny_{approval['id']}", "title": "Deny"}} ] } }) # Pause the run until approval or expiry await db.execute("UPDATE runs SET state = 'awaiting_approval' WHERE id = $1", run_id)
```

11. Channel Integration — WhatsApp & iMessage

11.1 WhatsApp Cloud API Integration

```
# WhatsApp Webhook Handler (Gateway Plane) from fastapi import FastAPI, Request, HTTPException import hmac, hashlib app = FastAPI() WHATSAPP_VERIFY_TOKEN = os.environ["WA_VERIFY_TOKEN"] WHATSAPP_APP_SECRET = os.environ["WA_APP_SECRET"] WHATSAPP_PHONE_ID = os.environ["WA_PHONE_NUMBER_ID"] WHATSAPP_API_URL = f"https://graph.facebook.com/v21.0/{WHATSAPP_PHONE_ID}/messages" @app.get("/webhooks/whatsapp") async def verify_webhook(request: Request): """WhatsApp webhook verification (GET request)."""" params = request.query_params if params.get("hub.verify_token") == WHATSAPP_VERIFY_TOKEN: return int(params["hub.challenge"]) raise HTTPException(403, "Invalid verify token") @app.post("/webhooks/whatsapp") async def receive_webhook(request: Request): """Process inbound WhatsApp message. MUST respond within 5 seconds."""" # 1. Validate HMAC signature body = await request.body() signature = request.headers.get("X-Hub-Signature-256", "") expected = "sha256=" + hmac.new(WHATSAPP_APP_SECRET.encode(), body, hashlib.sha256).hexdigest() if not hmac.compare_digest(signature, expected): raise HTTPException(403, "Invalid signature") # 2. Parse message data = await request.json() messages = extract_messages(data) # handles text, media, button replies, etc. for msg in messages: # 3. Dedup by message ID if await is_duplicate(msg.channel_msg_id): continue # 4. Send typing indicator IMMEDIATELY (< 500ms) await send_typing_indicator(msg.user_phone) # 5. Queue for async processing (DO NOT process in webhook handler) await message_queue.enqueue("process_message", msg.dict()) return {"status": "ok"} # Must return 200 within 5 seconds async def send_whatsapp_message(to: str, text: str): """Send a text message via WhatsApp Cloud API."""" # Split long messages (WhatsApp limit: 4096 chars) chunks = split_message(text, max_length=4000) for chunk in chunks: async with httpx.AsyncClient() as client: await client.post(WHATSAPP_API_URL, headers={ "Authorization": f"Bearer {os.environ['WA_ACCESS_TOKEN']}", "Content-Type": "application/json" }, json={ "messaging_product": "whatsapp", "to": to, "type": "text", "text": {"body": chunk} })
```

11.2 WhatsApp Template Messages (Proactive Outreach)

All proactive messages MUST use pre-approved templates (Meta requires 24-48h approval). Submit templates during Month 1.

Template Name	Category	Body	Buttons
morning_briefing	UTILITY	Good morning, {{1}}! Here's your day: {{2}}	[View Details]
meeting_prep	UTILITY	Your meeting with {{1}} starts in {{2}}. {{3}}	[Get Prep] [Reschedule]
follow_up_nudge	UTILITY	Reminder: you mentioned following up with {{1}} about {{2}}. It's been {{3}} days.	[Draft Follow-up] [Dismiss]
travel_alert	UTILITY	Heads up: traffic to {{1}} is {{2}}. Leave by {{3}} to arrive on time.	[Get Directions] [Reschedule]
approval_request	UTILITY	I'd like to {{1}}. {{2}}	[Approve] [Edit] [Deny]
task_complete	UTILITY	Done! {{1}}	[Details]
conflict_alert	UTILITY	Schedule conflict: {{1}} overlaps with {{2}}. {{3}}	[Fix It] [Ignore]
weekly_digest	UTILITY	Your week ahead: {{1}} meetings, {{2}} follow-ups pending. {{3}}	[View Week]

12. Tool/Connector Architecture

12.1 Connector Implementation Pattern

Every connector follows the same pattern. Here is the Google Calendar connector as the reference implementation:

```
# connectors/google_calendar.py from app.tools.base import BaseConnector, ToolSpec, ToolResult from
pydantic import BaseModel, Field from datetime import datetime from typing import Optional class
ListEventsInput(BaseModel): start_date: datetime end_date: datetime max_results: int = Field(default=10,
le=50) class CreateEventInput(BaseModel): title: str start_time: datetime end_time: datetime attendees:
list[str] = [] # email addresses location: Optional[str] = None description: Optional[str] = None class
GoogleCalendarConnector(BaseConnector): TOOLS = [ ToolSpec( name="google_calendar.list_events",
description="List calendar events in a date range. Returns: title, time, attendees, location.",
input_schema=ListEventsInput.model_json_schema(), output_schema={}, # dynamic risk_level=RiskLevel.NONE,
is_reversible=True, timeout_ms=5000, tags=["calendar", "read", "google"], ), ToolSpec(
name="google_calendar.create_event", description="Create a new calendar event with title, time, attendees,
and location.", input_schema=CreateEventInput.model_json_schema(), output_schema={},
risk_level=RiskLevel.LOW, is_reversible=True, # can delete the event
requires_approval_above=RiskLevel.LOW, timeout_ms=8000, tags=["calendar", "write", "google"], ), # ...
update_event, delete_event, find_free_slots ] async def execute(self, tool_name: str, args: dict, user_id:
UUID) -> ToolResult: account = await self.get_account(user_id, "google") credentials = await
self.decrypt_tokens(account) service = build_calendar_service(credentials) if tool_name ==
"google_calendar.list_events": events = service.events().list( calendarId='primary',
timeMin=args['start_date'].isoformat(), timeMax=args['end_date'].isoformat(),
maxResults=args['max_results'], singleEvents=True, orderBy='startTime' ).execute() # IMPORTANT: Compress
output. Raw API response is 5K+ tokens. # Extract only: title, start, end, attendees, location return
ToolResult( tool_name=tool_name, status="success", output={"events": [compress_event(e) for e in
events.get('items', [])]}, latency_ms=elapsed_ms() )
```

12.2 Connector Build Priority

Month	Connector	Tools	Risk Levels
1-2	**Google Calendar**	list_events, create_event, update_event, delete_event, find_free_slots	read=none, create/update=low, delete=high
1-2	**Google Gmail**	list_emails, search_emails, get_email, draft_email, send_email, label_email	read/search=none, draft=low, send=medium
2	**Google Contacts**	list_contacts, search_contacts, get_contact	all=none
2-3	**Web Search (Tavily)**	search_web, get_page_content	all=none
3-4	**Microsoft Calendar**	Same as Google Calendar	Same as Google
3-4	**Microsoft Outlook**	Same as Google Gmail	Same as Google
5-6	**Slack**	send_message, list_channels, get_channel_summary	read=none, send=medium
8-9	**Plaid (Finance)**	get_balances, get_transactions, get_spending_summary	all=high (read-only but sensitive)

13. Proactive Intelligence — Triggers & Scheduling

13.1 Value > Interruption Formula

```
async def should_send_proactive(trigger: ProactiveTrigger, user: User) -> tuple[bool, float]: """Evaluate whether a proactive notification is worth sending.""" action_importance = compute_importance(trigger) # 0.0-1.0 time_sensitivity = compute_urgency(trigger) # 0.0-1.0 user_relevance = compute_relevance(trigger, user) # 0.0-1.0 # Interruption cost based on time of day and channel now = datetime.now(tz=user.timezone) if is_quiet_hours(now, user.quiet_hours): interruption_cost = 3.0 # very high during quiet hours elif is_focused_work(now, user): # in a meeting or deep work block interruption_cost = 2.0 else: interruption_cost = 1.0 # Channel cost multiplier channel_cost = 1.5 if trigger.channel == "push" else 1.0 # push is more interruptive interruption_cost *= channel_cost # Compute notification value value = (action_importance * time_sensitivity * user_relevance) / interruption_cost # Anti-annoyance checks if trigger.user_dismissed_count > 3 and trigger.fire_count > 5: value *= 0.5 # user keeps dismissing this trigger type daily_fires = await count_proactive_today(user.id) if daily_fires >= user_max_daily_proactive(user): return False, value # hit daily cap threshold = trigger.confidence_threshold # default 0.7 return value >= threshold, value
```

13.2 Built-in Trigger Implementations

Trigger	Condition	Action	Default Schedule
Morning Briefing	30min before first calendar event	Summarize: meetings, flagged emails, weather, reminders	Daily, personalized to user's first event
Pre-Meeting Prep	30min before any meeting	Fetch attendee context from email + memory. Offer summary.	Per-meeting with >1 attendee
Follow-Up Nudge	User committed to follow up; N days elapsed	Suggest drafting a follow-up email	Check daily; fire after 3 days default
Travel Alert	60min before meeting with location	Check traffic API; alert if leave-by time approaching	Per-meeting with location field
Conflict Detection	On calendar sync; overlap detected	Alert user with resolution options	On every delta-sync
Deadline Warning	Task/event deadline approaching	Remind user with time remaining	24h, 4h, 1h before deadline
Weekly Digest	Sunday evening or Monday morning	Summarize upcoming week: meetings, deadlines, follow-ups	Weekly

14. Conversation Design System — Persona & Patterns

The conversation design system ensures the agent communicates consistently and effectively across all interactions. It defines the persona, tone rules, and message templates.

14.1 Persona Specification (Injected in Every System Prompt)

Agent Persona: 'Exec'

Core traits: Competent, warm, concise, proactive.
Mental model: An exceptional executive assistant with 3 years of working with this specific user.
Communication style: Lead with the answer. Never use filler phrases. Keep responses under 200 words. Own mistakes directly.
Tone calibration: Professional but not stiff. Friendly but not casual. Confident but not arrogant. Direct but not curt.
Forbidden phrases: 'Certainly!', 'Of course!', 'I'd be happy to...', 'Great question!', 'As an AI...'
Error style: 'I got the date wrong -- here's the corrected version.' NOT 'I sincerely apologize for the confusion...'

14.2 20 Core Interaction Patterns

#	Pattern	Structure	Example
1	Task completion	[Result]. [One-line context].	'Scheduled your 1:1 with Sarah for Tuesday 10am. She's confirmed.'
2	Approval request	[Proposed action]. [Why asking]. [Buttons].	'Send this draft to John Chen? He's a new contact.' [Send][Edit][Cancel]
3	Clarification	[What understood]. [What's ambiguous]. [Options].	'Reschedule which meeting -- your 10am standup or 2pm client call?'
4	Error recovery	[What went wrong]. [What I'm doing]. [What you can do].	'Google Calendar isn't responding. Retrying in 30s.'
5	Proactive alert	[Key info]. [Why it matters]. [Suggested action].	'Traffic is heavy. Leave by 2:15 for your 3pm meeting.'
6	Morning briefing	[Greeting]. [Calendar]. [Flagged items]. [Weather].	'Morning! 3 meetings today, first at 10am. 2 urgent emails. 52F, clear.'
7	Learning moment	[What I learned]. [Future benefit].	'Got it -- keeping drafts shorter from now on.'
8	Don't know	[Can't find it]. [Where I looked]. [Suggestion].	'No contact named Mike in your email or calendar. More context?'
9	Earned autonomy	[What I did]. [Why I could]. [How to change].	'Scheduled the standup -- you've approved 15 similar requests.'
10	Multi-step progress	[Step completed]. [What's next]. [ETA].	'Found Sarah's email. Drafting reply now... 10 seconds.'

15. Context Compiler — Token Budget Management

```
class ContextCompiler: """Assembles the LLM context window within tier-specific token budgets.""" BUDGETS = { Tier.INSTANT: {}, # No LLM call Tier.SINGLE_SHOT: { "system_prompt": 800, "user_profile": 400, "conversation": 500, "memories": 300, "tool_schemas": 400, "tool_results": 500, "output_reserve": 500 }, # Total: 3,400 Tier.REACT: { "system_prompt": 1200, "user_profile": 800, "conversation": 2000, "memories": 1000, "tool_schemas": 1000, "rag_chunks": 2000, "tool_results": 2000, "output_reserve": 1000 }, # Total: 11,000 Tier.MULTI_STEP: { "system_prompt": 2000, "user_profile": 1500, "conversation": 4000, "memories": 2000, "tool_schemas": 2000, "rag_chunks": 6000, "tool_results": 4000, "output_reserve": 2000 }, # Total: 23,500 } async def compile(self, user_id: UUID, conversation_id: UUID, message: str, classification: IntentClassification) -> dict: budget = self.BUDGETS[classification.tier] context = {} # 1. System prompt (static prefix, benefits from OpenAI prompt caching) context["system"] = self.build_system_prompt(user_id, budget["system_prompt"]) # 2. User profile + preferences (semi-static, changes rarely) context["profile"] = await self.get_user_profile(user_id, budget["user_profile"]) # 3. Conversation history (sliding window + summary) context["history"] = await self.get_conversation_history( conversation_id, budget["conversation"] ) # 4. Relevant memories (intent-filtered retrieval) context["memories"] = await self.retrieve_memories( user_id, message, classification, budget.get("memories", 0) ) # 5. Tool schemas (only tools relevant to this intent) context["tools"] = self.get_tool_schemas( classification.required_tools, budget.get("tool_schemas", 0) ) # Log token utilization for monitoring total_used = sum(count_tokens(v) for v in context.values()) total_budget = sum(budget.values()) log_context_utilization(classification.tier, total_used, total_budget) return context
```

16. Workflow Orchestration — Temporal Patterns

Temporal is used **only for T3 (multi-step) workflows**. T0-T2 use simple async processing. This saves significant infrastructure cost while preserving Temporal's value for complex, long-running tasks that need checkpointing and retry.

16.1 When to Use Temporal vs. Simple Async

Scenario	Orchestration	Rationale
T0: Instant response	Inline (no async)	No processing needed; template response
T1: Single tool call	asyncio.create_task	One LLM call + one tool call. No checkpointing needed.
T2: ReAct loop	BullMQ job	May take 10-30s. Job queue provides retry + dead-letter. No checkpointing.
T3: Multi-step plan	**Temporal workflow**	Long-running (30s-5min). Needs checkpointing, compensation, and auditable history.
Background: delta-sync	BullMQ recurring job	Periodic polling. Simple retry on failure.
Background: proactive trigger	BullMQ scheduled job	Scheduled event. Fire-and-forget with retry.
Background: nightly consolidation	BullMQ job	Batch processing. Idempotent. Simple retry.

16.2 T3 Temporal Workflow Definition

```
# workflows/multi_step_plan.py from temporalio import workflow, activity from datetime import timedelta @workflow.defn class MultiStepPlanWorkflow: """T3 workflow: plan, critique, execute with checkpoints.""" @workflow.run async def run(self, run_id: str, user_id: str, message: str, envelope: dict): # Step 1: Generate plan plan = await workflow.execute_activity( generate_plan, args=[user_id, message, envelope], start_to_close_timeout=timedelta(seconds=30), retry_policy=RetryPolicy(maximum_attempts=2) ) # Step 2: Critic review critique = await workflow.execute_activity( critique_plan, args=[plan, envelope], start_to_close_timeout=timedelta(seconds=15), ) if not critique["approved"]: # Re-plan with critic feedback (max 3 attempts) for attempt in range(3): plan = await workflow.execute_activity( revise_plan, args=[plan, critique["feedback"], envelope], start_to_close_timeout=timedelta(seconds=30), ) critique = await workflow.execute_activity( critique_plan, args=[plan, envelope], start_to_close_timeout=timedelta(seconds=15), ) if critique["approved"]: break else: # Escalate to user after 3 failed attempts return await escalate_to_user(run_id, plan, critique) # Step 3: Execute plan steps with checkpoints results = [] for step in plan["steps"]: # Check if approval needed if step["requires_approval"]: approved = await workflow.execute_activity( request_and_wait_approval, args=[run_id, step], start_to_close_timeout=timedelta(minutes=10), ) if not approved: continue # skip this step # Execute tool result = await workflow.execute_activity( execute_tool, args=[step["tool"], step["args"], envelope], start_to_close_timeout=timedelta(seconds=step.get("timeout_s", 10)), retry_policy=RetryPolicy(maximum_attempts=2) ) results.append(result) # Send progress update to user await workflow.execute_activity( send_progress, args=[run_id, step, result], start_to_close_timeout=timedelta(seconds=5), ) # If step failed, attempt re-plan from this checkpoint if result["status"] == "failed" and step.get("critical", False): return await replan_from_checkpoint(run_id, plan, results, envelope) return {"status": "completed", "results": results}
```

17. Security Implementation

17.1 Security Controls Matrix

Control	Implementation	When	Code Reference
Input sanitization	Strip potential prompt injection patterns from user messages before LLM context. Remove markdown injection, system prompt overrides, role-play attacks.	Every inbound message	gateway/sanitizer.py
Output validation	Every LLM response validated against Pydantic schema (structured output) or content policy (free text). Reject and retry on validation failure.	Every LLM call	brain/guardrails.py
Content moderation	OpenAI Moderation API check on all user inputs and LLM outputs. Block flagged content. Log for review.	Every message	brain/moderation.py
Token encryption	OAuth tokens encrypted with AES-256-GCM. Encryption key in AWS Secrets Manager. Tokens never logged or passed to LLM.	Token storage/retrieval	hands/oauth_vault.py
Idempotency	Every tool execution has a unique idempotency key = SHA256(run_id + step_id + tool + canonical_args). DB unique constraint prevents duplicates.	Every tool call	hands/executor.py
Row-Level Security	Postgres RLS on all tables. app.user_id session variable set on every connection. No cross-user data access possible.	Every DB query	db/middleware.py
Rate limiting	Per-user: 60 msgs/min (Redis sliding window). Per-IP: 100 req/min. Gradual degradation, not hard cutoff.	Every request	gateway/rate_limiter.py
Webhook validation	WhatsApp: HMAC-SHA256 signature verification. Apple: certificate chain validation.	Every webhook	gateway/webhooks.py
PII redaction	Redact email addresses, phone numbers, and names from logs and LLM observability. Keep in DB only.	All logging	core/redactor.py
Secret rotation	OAuth refresh tokens auto-rotated on use. API keys rotated every 90 days via Secrets Manager.	Background job	hands/key_rotation.py

18. Observability & Monitoring

18.1 Key Metrics Dashboard

Metric	Target	Alert Threshold	Source
p95 latency (WhatsApp e2e)	< 3s	> 5s for 5min	OTEL traces
p95 latency (first response)	< 2s	> 3s for 5min	OTEL traces
Error rate (5xx)	< 0.1%	> 1% for 5min	HTTP status codes
LLM call success rate	> 99.5%	< 98% for 10min	LLM Router logs
Tool execution success rate	> 98%	< 95% for 10min	tool_executions table
Cost per interaction (blended)	< $0.03	> $0.05 for 1hr	Cost tracking
Tier distribution	T0:15% T1:55% T2:25% T3:5%	T3 > 10% for 1hr	Tier Router logs
Semantic cache hit rate	> 20%	< 10% for 1day	Cache metrics
Memory retrieval latency	< 50ms	> 200ms for 10min	Memory Engine traces
Approval response time	< 2min median	> 5min median for 1hr	Approvals table
Proactive message helpful rate	> 50%	< 30% for 1wk	Feedback signals
DAU / MAU ratio	> 50%	< 30% for 2wks	Analytics

19. Testing Strategy — Unit to Red-Team

Test Type	What It Validates	Tool/Framework	Run Frequency	Blocking?
Unit tests	Individual functions: context compiler, memory retrieval, tier router, autonomy scoring, channel adapter	pytest + hypothesis (property-based)	Every PR	Yes
Integration tests	Service-to-service: Gateway->Brain, Brain->Hands, DB queries, Redis ops	pytest + testcontainers	Every PR	Yes
Contract tests	API contracts: Pydantic schemas, OpenAPI spec, tool schemas backward-compatible	schemathesis + custom validators	Every PR that changes schemas	Yes
Connector tests	Each tool: golden input/output pairs. Record/replay for API responses. Handles errors, timeouts, rate limits.	pytest + VCR.py (recorded fixtures)	Every PR + nightly full suite	Yes
Agentic eval	End-to-end conversations: 50 golden scenarios covering all tiers, intents, and edge cases. Scored by LLM-as-judge.	Braintrust eval framework	Nightly + before deploy	Yes (if score drops)
Red-team eval	Adversarial inputs: prompt injection, jailbreak, data exfiltration, wrong recipient, hallucination probing	Custom red-team harness + HarmBench	Weekly	Yes (if new vulnerability found)
Load test	Concurrent users: 100, 1K, 10K simultaneous sessions. Latency under load.	k6 + Grafana	Before each phase milestone	No (informational)
Chaos test	Failure modes: kill Brain plane, kill Redis, throttle Postgres, LLM timeout. Verify graceful degradation.	Custom fault injection	Monthly	No (informational)

20. Deployment Architecture — AWS Production

20.1 Infrastructure Layout

```
# AWS infrastructure (SST / Pulumi definition) # # VPC: 10.0.0.0/16 # Public subnets: 10.0.1.0/24, 10.0.2.0/24 (ALB, NAT Gateway) # Private subnets: 10.0.10.0/24, 10.0.11.0/24 (ECS tasks, RDS) # # ECS Cluster: "executive-os-prod" # Service: gateway (2 tasks, 0.5 vCPU, 1GB RAM) # Service: brain (2 tasks, 1 vCPU, 2GB RAM) # Service: hands (2 tasks, 0.5 vCPU, 1GB RAM) # Service: workers (1 task, 0.5 vCPU, 1GB RAM) -- BullMQ workers # # RDS: PostgreSQL 16 (db.t4g.medium, Multi-AZ, 100GB gp3) # ElastiCache: Redis 7 (cache.t4g.small, single-node) # ALB: Application Load Balancer (webhook termination + TLS) # S3: Attachments + backups # Secrets Manager: OAuth tokens, API keys # CloudWatch: Log groups per service # # Estimated monthly cost at 1K users: # ECS Fargate: ~$100 # RDS Postgres: ~$60 # ElastiCache: ~$25 # ALB: ~$20 # S3 + transfer: ~$10 # Secrets Manager: ~$10 # CloudWatch: ~$15 # Total infra: ~$240/month (+ LLM API costs)
```

20.2 CI/CD Pipeline

Stage	Trigger	Actions	Gate
Build	PR opened/updated	Lint (ruff), type check (mypy), unit tests, contract tests	All pass
Integration	PR approved	Integration tests with testcontainers, connector record/replay tests	All pass
Staging deploy	Merge to main	Deploy to staging ECS cluster, run agentic evals (50 scenarios)	Eval score >= baseline
Canary deploy	Staging passes	Deploy to 10% of prod traffic, monitor for 30min	No error rate spike
Full deploy	Canary passes	Rolling deploy to 100% of prod traffic	Automatic
Rollback	Error rate > 1% for 5min	Automatic rollback to previous task definition	Automatic

21. Onboarding Flow — Zero to Wow in 5 Minutes

21.1 Step-by-Step Flow

Step	Time	User Action	Agent Action	Data Captured
1	0:00	Sends any message to WhatsApp number	Responds: 'Hey! I'm Exec, your personal assistant. What should I call you?'	User record created
2	0:30	Says their name	Responds: 'Nice to meet you, {name}! Let me connect to your calendar so I can start helping.' [Connect Google]	display_name stored
3	1:00	Taps OAuth button, authenticates	Immediately reads tomorrow's calendar. 'You have 4 meetings tomorrow. First at 9am with Sarah. Want a prep summary?'	Google tokens, calendar events synced
4	2:00	Says 'sure' or asks a question	Delivers meeting prep or answers question. Then: 'Quick setup: what are your work hours?'	First run, first episode
5	3:00	Answers work hours question	Confirms and asks: 'Brief or detailed responses?' [Brief] [Detailed]	work_hours, communication preference
6	4:00	Selects preference	Adapts response style. 'All set! Connect your email too?' [Connect Gmail]	preferences stored
7	5:00	Taps email OAuth	Scans last 7 days async. 'I found 3 unanswered emails. Want me to draft replies?'	Email synced, contacts extracted

22. Cost Model & Unit Economics

22.1 Per-Interaction Cost Breakdown

Component	T0 (15%)	T1 (55%)	T2 (25%)	T3 (5%)	Blended
LLM: Classification	$0	$0.001	$0.001	$0.001	$0.0008
LLM: Main inference	$0	$0.003	$0.040	$0.150	$0.0110
LLM: Embeddings	$0	$0.0002	$0.0005	$0.001	$0.0003
Tool API calls	$0	$0.001	$0.003	$0.010	$0.0017
Compute (amortized)	$0.0005	$0.001	$0.002	$0.005	$0.0013
Database queries	$0.0001	$0.0003	$0.001	$0.002	$0.0004
TOTAL	**$0.0006**	**$0.0065**	**$0.048**	**$0.169**	**$0.017**

Blended Cost: $0.017/interaction

With 20% overhead for retries, caching infra, and logging: **~$0.021/interaction**. At 50 interactions/user/day and $19.99/month subscription: **93% gross margin at 10K users.**

23. Build Plan — Sprint-Level Detail

Each phase delivers a **production-quality** milestone. No throwaway code. Every component built in Phase 1 is still running in production.

PHASE 1: Foundation

M1 Sprint 1-2: Infrastructure: AWS VPC, ECS cluster, RDS Postgres + pgvector, Redis. CI/CD pipeline. Monitoring (Axiom + OTEL).

M1 Sprint 3-4: Gateway Plane: WhatsApp webhook receiver, signature validation, message dedup, typing indicator, session manager (Redis).

M1-2 Sprint 5-8: Google Calendar connector: list_events, create_event, update_event, delete_event, find_free_slots. OAuth flow. Delta-sync (2min).

M2 Sprint 9-10: Brain Plane v1: Tier Router (T0 + T1 only), Intent classifier (GPT-4o-mini), Context compiler (T1 budget), Response generator.

M2 Sprint 11-12: Google Gmail connector: list_emails, search_emails, get_email, draft_email. Delta-sync (5min).

M2-3 Sprint 13-14: Memory Engine v1: Structured profile store, episodic memory after each run, pgvector semantic search, retrieval pipeline.

M3 Sprint 15-16: ACE Engine v1: action classification, approval flow (WhatsApp buttons), cold-start priors, Beta parameter updates.

M3 Sprint 17-18: Conversation design: persona in system prompt, 10 core message patterns, error communication templates.

PHASE 2: Intelligence

M4 Sprint 1-2: Tier 2 (ReAct loop): multi-step reasoning with GPT-4o. Max 5 iterations. Complexity escalation from T1.

M4 Sprint 3-4: Email sending: approval flow for send_email. Recipient verification (fuzzy match + 'Did you mean?'). Draft-review-send cycle.

M4-5 Sprint 5-8: Microsoft 365 connector: Calendar + Outlook + Contacts. Cross-platform scheduling.

M5 Sprint 9-10: Proactive Intelligence Engine: morning briefing, pre-meeting prep, follow-up nudges. WhatsApp templates submitted and approved.

M5-6 Sprint 11-14: Web search connector (Tavily). Research queries. Tier 3 (multi-step planning): Temporal workflow, plan generation + critic + checkpoints.

M6 Sprint 15-16: Memory improvements: implicit preference learning, contact graph enrichment, nightly consolidation, confidence decay.

M6 Sprint 17-18: Agentic eval suite: 50 golden scenarios in Braintrust. LLM-as-judge scoring. CI blocking gate.

PHASE 3: Expansion

M7 Sprint 1-4: Apple Messages for Business: webhook integration, iMessage-native UX (time pickers, rich links), template adaptation.

M7 Sprint 5-6: Slack connector: read/send messages, channel summaries.

M7-8 Sprint 7-10: Advanced scheduling: multi-person availability, timezone intelligence, travel time estimation, conflict resolution.

M8-9 Sprint 11-14: Semantic caching deployed. Prompt caching tuned. Model cascade metrics analyzed and optimized from production data.

M9 Sprint 15-16: Financial connector (Plaid): get_balances, get_transactions, get_spending_summary. High-risk approval flow.

M9 Sprint 17-18: Red-team eval: prompt injection corpus, wrong recipient traps, data exfil attempts. Fix all vulnerabilities found.

PHASE 4: Polish & Scale

M10 Sprint 1-4: Travel connectors, smart home basics, Notion/Google Docs search.

M10-11 Sprint 5-8: Advanced proactive intelligence: pattern detection, smart reminders, deadline tracking, spending alerts.

M11 Sprint 9-12: Performance: p95 < 3s on WhatsApp. Cost < $0.03/interaction. Semantic cache > 20% hit rate.

M11-12 Sprint 13-14: Load test at 10K concurrent users. Graceful degradation implemented and verified.

M12 Sprint 15-16: Security hardening: pentest, prompt injection testing, OWASP LLM Top 10 audit.

M12 Sprint 17-18: Production launch prep: runbooks, incident playbooks, on-call rotation, documentation.

24. Team Structure & Ownership Matrix

Role	#	Owns	Key Skills	Hire By
Tech Lead / Architect	1	Architecture, Brain Plane, code review, system design	LLM orchestration, distributed systems, Python	Day 1
Backend (Brain)	2	LLM integration, planner, memory engine, context compiler, prompts	Python, OpenAI API, pgvector, NLP	Day 1
Backend (Hands)	1	Connectors, OAuth, delta-sync, tool execution, background jobs	Python, REST APIs, OAuth, Google/Microsoft APIs	Month 1
Backend (Gateway)	1	WhatsApp/iMessage, sessions, channel adapters, rate limiting	Python/TS, WhatsApp Cloud API, Apple Business Chat	Month 1
DevOps / Infra	1	AWS deployment, CI/CD, monitoring, DB, cost optimization	AWS ECS, Postgres, Redis, SST/Terraform, OTEL	Month 1
Product / Conv. Designer	1	Persona, conversation patterns, onboarding, proactive UX, evals	UX writing, conversation design, prompt engineering	Month 2

Appendix A: Environment Variables

```
# === Gateway === WA_VERIFY_TOKEN= # WhatsApp webhook verification token WA_APP_SECRET= # WhatsApp app secret (HMAC validation) WA_ACCESS_TOKEN= # WhatsApp Cloud API access token WA_PHONE_NUMBER_ID= # WhatsApp business phone number ID WA_BUSINESS_ACCOUNT_ID= # WhatsApp business account ID APPLE_BUSINESS_CHAT_ID= # Apple Messages for Business ID CLERK_SECRET_KEY= # Clerk auth secret # === Brain === OPENAI_API_KEY= # OpenAI API key (GPT-4o + GPT-4o-mini + embeddings) OPENAI_ORG_ID= # OpenAI organization ID # === Hands === GOOGLE_CLIENT_ID= # Google OAuth client ID GOOGLE_CLIENT_SECRET= # Google OAuth client secret MICROSOFT_CLIENT_ID= # Microsoft OAuth client ID MICROSOFT_CLIENT_SECRET= # Microsoft OAuth client secret TAVILY_API_KEY= # Tavily web search API key # === Data === DATABASE_URL= # postgresql://user:pass@host:5432/executive_os REDIS_URL= # redis://host:6379/0 # === Infrastructure === AWS_REGION=us-east-1 AWS_SECRETS_PREFIX=executive-os/prod/ S3_BUCKET=executive-os-attachments AXIOM_API_TOKEN= # Observability platform token AXIOM_DATASET=executive-os-prod # === Feature Flags === FEATURE_TIER_3_ENABLED=true FEATURE_PROACTIVE_ENABLED=true FEATURE_IMESSAGE_ENABLED=false # Enable in Phase 3
```

Appendix B: Error Codes & Taxonomy

Code	Category	User-Facing Message	Internal Action
E1001	Auth	'Please reconnect your Google account.'	OAuth token expired, refresh failed. Prompt re-auth.
E1002	Auth	'Please reconnect your Microsoft account.'	Same as above for Microsoft.
E2001	Tool	'Google Calendar is slow. Retrying...'	Tool timeout. Retry with backoff.
E2002	Tool	'I couldn't complete that action. Let me try another way.'	Tool failed permanently. Attempt alternative.
E2003	Tool	'That service is temporarily unavailable. I'll queue this.'	Circuit breaker open. Queue for retry.
E3001	LLM	'Let me think about that differently.'	LLM returned invalid output. Retry with different prompt.
E3002	LLM	'I'm having trouble processing that. Can you rephrase?'	LLM failed after 3 retries. Escalate to user.
E4001	Budget	'This task is more complex than usual. Continue?'	Run approaching budget cap. Ask before exceeding.
E5001	Safety	'I can't do that -- it goes against my safety guidelines.'	Content moderation flagged. Log incident.
E6001	Rate limit	'I'm getting a lot of requests. Give me a moment.'	User rate limit hit. Slow response, don't block.

Appendix C: WhatsApp Template Library

All templates below must be submitted to Meta for approval **before** proactive features launch. Submit during Month 1. Allow 24-48 hours for review.

Template Name	Category	Header	Body (with {{variables}})	Buttons
exec_morning_brief	UTILITY	none	Good morning, {{1}}! Here's your day:\n\n{{2}}	Quick Reply: 'Tell me more'
exec_meeting_prep	UTILITY	none	Your meeting with {{1}} is in {{2}}. Key context:\n\n{{3}}	Quick Reply: 'Full prep' | 'Thanks'
exec_followup	UTILITY	none	Hey {{1}}, reminder: you wanted to follow up with {{2}} about {{3}}. It's been {{4}} days.	Quick Reply: 'Draft it' | 'Dismiss'
exec_travel	UTILITY	none	Heads up: traffic to {{1}} is {{2}}. Leave by {{3}} to arrive on time.	Quick Reply: 'Directions' | 'Reschedule'
exec_conflict	UTILITY	none	Schedule conflict: {{1}} overlaps with {{2}} on {{3}}.	Quick Reply: 'Fix it' | 'Keep both'
exec_approval	UTILITY	none	I'd like to: {{1}}\n\nReason: {{2}}	Quick Reply: 'Approve' | 'Edit' | 'Deny'
exec_complete	UTILITY	none	Done! {{1}}	Quick Reply: 'Details'

Template Name	Category	Header	Body (with {{variables}})	Buttons
exec_weekly	UTILITY	none	Week ahead for {{1}}:\n\n{{2}} meetings\n{{3}} follow-ups pending\n\n{{4}}	Quick Reply: 'View week'

END OF DEVELOPER BLUEPRINT

Executive OS — 10x Redesign — February 2026